

PHP / PHP · PDO · OOP

Personen speichern und wieder auslesen

10 kleine Schritte · Ein gemeinsames Projekt · Lösungen getrennt

Das Ziel

Eine MySQL-/MariaDB-Datenbank mit nur einer Tabelle. Du liest Personen aus, erzeugst pro Datensatz ein Person-Objekt und zeigst die Werte in einer HTML-Tabelle. Ein Formular ergänzt neue Personen.

Spalte	Typ / Zweck
id	Ganze Zahl; Primärschlüssel; wird automatisch vergeben.
vorname	Text mit höchstens 50 Zeichen; Pflichtfeld im Formular.
nachname	Text mit höchstens 50 Zeichen; Pflichtfeld im Formular.

So startest du

1. Repository PHP_AI_DEMO unter htdocs ablegen. Apache und MySQL starten.
 2. datenbank.sql einmal in phpMyAdmin importieren. config.php an deine Zugangsdaten anpassen.
 3. Die Webeinführung öffnen; danach demo/lesen.php und demo/index.php ansehen.
- Einstieg im Repository: Optionales_Zusatzmaterial/db/index.html

So arbeitest du

Alle zehn Aufgaben bauen im selben Ordner aufgaben/ aufeinander auf. Bearbeite die nummerierten TODOs. HTML, Styles und der bekannte Session-Rahmen sind vorbereitet. Die Demos sind vollständig; die Aufgabenfassung ist es noch nicht.

Für alle Aufgaben

Zwei Pflichtfelder: Vorname und Nachname. Jeden POST-Eintrag vor dem Auslesen prüfen. Erst beide Angaben vollständig validieren, dann schreiben. Bei Fehlern bleiben gültige neue Eingaben erhalten; ungültige Felder werden geleert. Kein INSERT bei ungültigen Daten.

Nur erfundene Daten. Kein Login, keine Vererbung, kein MVC, keine weiteren Tabellen und kein UPDATE/DELETE. Demo, Aufgaben und Lösungen verwenden dieselbe DB. Ein erneuter SQL-Import fügt die Beispiele nochmals ein; er löscht nichts. PHP 8 mit pdo_mysql und mbstring wird benötigt.

01 Die kleine Datenbank importieren

Eine Datenbank · eine Tabelle · drei Spalten

Dateien: datenbank.sql / phpMyAdmin

1. Starte Apache und den Datenbankdienst in XAMPP. Importiere datenbank.sql einmal in phpMyAdmin.
2. Öffne PHP _personen und die Tabelle personen. Lies die drei Beispielpersonen.
3. Ordne die Spalten id, vorname und nachname zu. Erkläre, warum die ID kein Eingabefeld im Formular braucht.

Teste selbst

Beim ersten Import in eine neue Datenbank sind drei Zeilen vorhanden.

Notiere die Spaltentypen. Wiederhole den Import nicht: Die Beispieldaten würden erneut eingefügt.

02 PHP mit der Datenbank verbinden

PDO · Zugangsdaten · Verbindungsfehler

Dateien: config.php / aufgaben/db.php

1. Prüfe in config.php Host, Port, Datenbankname, Benutzer und Passwort für deine lokale Installation.
2. Ergänze TODO 02: Gib ein PDO-Objekt zurück. Verwende den vorbereiteten DSN und das Options-Array für Exceptions und echte Prepared Statements.
3. Entferne die vorläufige Exception. Die Fehlerbehandlung der aufrufenden Seite ist bereits vorhanden.

Teste selbst

Öffne aufgaben/index.php. Mit richtigen Zugangsdaten verschwindet der Verbindungsfehler.

Ein absichtlich falscher Datenbankname erzeugt einen verständlichen Hinweis. Stelle danach den richtigen Namen wieder her.

03 Die Klasse Person ergänzen

Daten als Objekt · Konstruktor · Getter

Dateien: aufgaben/Person.php / objekt_test.php

1. Ergänze drei private Attribute für ID, Vorname und Nachname. Verwende passende Datentypen.
2. Übernimm die Konstruktorparameter in die Attribute. Ergänze die drei Getter und ersetze deren vorläufige Rückgabewerte.
3. Die Klasse enthält keine Datenbankverbindung und keine HTML-Ausgabe. Öffne den vorgegebenen Objekttest.

Teste selbst

objekt_test.php zeigt unter Ist: 7 / Mia / Müller.

Ändere die Testwerte im Aufruf. Die Getter liefern die neuen Werte. Es wird dabei noch nichts gespeichert.

04 Datensätze in Person-Objekte umwandeln

SELECT · fetch · ein Objekt pro Zeile

Dateien: aufgaben/index.php, PHP-Bereich vor dem HTML

1. Lies id, vorname und nachname aus personen, aufsteigend nach id sortiert. Ergänze TODO 04.
2. Hole die Ergebniszeilen nacheinander als assoziative Arrays ab.
3. Erzeuge innerhalb der Schleife für jede Zeile ein eigenes Person-Objekt. Sammle die Objekte in \$personen. Übernimm die ID als int.

Teste selbst

Nach dem ersten Import enthält \$personen drei Objekte der Klasse Person.

Kontrolliere die Variable im Debugger oder vorübergehend mit var_dump(). Entferne Debug-Ausgaben anschließend wieder.

05 Die Objekte als HTML-Tabelle anzeigen

foreach · Getter · HTML-Ausgabe

Dateien: aufgaben/index.php, Tabellenkörper

1. Ergänze TODO 05: Durchlaufe das Array \$personen mit foreach.
2. Erzeuge pro Person eine Tabellenzeile. Lies alle drei Werte über die Getter; gib die Namen mit dem vorgegebenen html()-Helper aus.
3. Zeige bei einer leeren Objektliste eine verständliche Meldung über alle drei Spalten an.

Teste selbst

Jede Beispielperson erscheint genau einmal mit ihrer ID.

Simuliere vorübergehend eine leere Liste und prüfe die Leermeldung. Entferne die Teständerung wieder.

06 Die beiden POST-Einträge entgegennehmen

Vor dem Auslesen: isset und Texttyp

Dateien: aufgaben/speichern.php

1. Das Formular ist fertig vorbereitet. Ergänze in TODO 06 für jeden POST-Eintrag eine eigene Prüfung mit isset() und is_string().
2. Fehlt ein Eintrag oder ist er kein Textwert, sammle eine passende Feldmeldung. Verarbeite das andere Feld trotzdem.
3. Entferne bei vorhandenen Textwerten äußere Leerzeichen. Lege die Werte zunächst in \$vorname und \$nachname ab, noch nicht in der Datenbank.

Teste selbst

Untersuche die Variablen am Haltepunkt nach dem Empfang: Mia / Müller kommt in den Textvariablen an.

Entferne im Browser testweise das name-Attribut eines Feldes oder sende einen Array-Wert. Es darf keine Undefined-array-key-Warnung entstehen.

07 Inhalte prüfen und bei Fehlern zurückleiten

Pflichtfelder · maximal 50 Zeichen · Werterhalt

Dateien: aufgaben/speichern.php, TODO 07a und 07b

1. Prüfe beide noch nicht beanstandeten Texte: nach trim() nicht leer, gültiges UTF-8 und höchstens 50 Zeichen. Die Zeichenfunktionen sind als Baustein vorgegeben.
2. Sammle alle Fehler. Übernimm ausschließlich gültige Eingaben in \$werte. Entferne den vorläufigen TODO-Schutz erst nach vollständiger Prüfung.
3. Bei Fehlern: Fehler und gültige Werte unter \$_SESSION["formular"] ablegen, mit 303 zurückleiten und beenden. Die Anzeige und das Wiedereinsetzen sind schon vorbereitet.

Teste selbst

Mia / leer: Vorname bleibt, Nachname ist leer, keine neue DB-Zeile.

Beide leer: beide Feldmeldungen. Leerzeichen oder 51 Zeichen: betroffene Eingabe wird abgelehnt.

08 Eine gültige Person mit INSERT speichern

prepare · Platzhalter · execute

Dateien: aufgaben/speichern.php, TODO 08

1. Ergänze nach erfolgreicher Prüfung eine INSERT-Anweisung mit zwei benannten Platzhaltern. Die ID wird nicht von dir übergeben.
2. Bereite die Anweisung mit prepare() vor. Übergib beide Werte getrennt vom SQL-Text an execute().
3. Entferne die vorläufige Exception. Setze niemals Formulardaten durch Verkettung direkt in den SQL-Befehl ein.

Teste selbst

Speichere einmal Nora / Weber und prüfe in phpMyAdmin: Es gibt eine zusätzliche Zeile.

Ohne Aufgabe 09 erscheint nach dem INSERT noch der Übergangshinweis. Nicht erneut absenden; öffne index.php per GET.

09 Nach dem Speichern die aktuelle Liste laden

POST → INSERT → 303 → GET → SELECT

Dateien: aufgaben/speichern.php, TODO 09

1. Hinterlege nach dem erfolgreichen INSERT eine kurze Erfolgsmeldung in `$_SESSION["erfolg"]`.
2. Ersetze die vorläufige Ausgabe durch die 303-Rückleitung zu `index.php` und ein anschließendes `exit`.
3. Die neue GET-Anfrage soll die Personen erneut aus der Datenbank lesen und die ergänzte Tabelle zeigen. Die Erfolgsmeldung wird einmalig angezeigt.

Teste selbst

Formular absenden: Person in der Liste, Erfolgsmeldung sichtbar, Eingabefelder leer.

F5 auf der zurückgeleiteten Seite erzeugt keine weitere Zeile. Bewusstes erneutes Absenden ist dagegen ein neues INSERT.

10 Den gesamten Ablauf prüfen

Zusammenführen · Sonderzeichen · Fehlerfälle

Dateien: Alle Dateien im Aufgabenordner

1. Prüfe einen gültigen Namen, leere und fehlende Werte, Arrays und beide Felder falsch. Für den Test mit 51 Zeichen entfernst du im Browser-Inspector vorübergehend `maxlength`; für einen fehlenden Wert das `name`-Attribut.
2. Speichere erfundene Testnamen mit Umlaut, Bindestrich und Apostroph. Prüfe außerdem, dass HTML-Zeichen als Text erscheinen.
3. Öffne die Übersicht in einem privaten Browserfenster: gespeicherte Personen müssen auch ohne die bisherige Session aus der Datenbank gelesen werden.

Teste selbst

Ungültige Eingaben erzeugen keine neue Zeile; gültige neue Eingaben bleiben nach Fehlern erhalten.

Erkläre den Weg eines Datensatzes: DB-Zeile → Person-Objekt → Getter → HTML. Ein Objekt allein speichert nichts in MySQL.

Merkhilfe für den Datenbank-Einstieg

Bausteine zum Nachschlagen – keine vollständigen Aufgabenlösungen.

Die vier beteiligten Teile

Browser: Formular und Tabelle. PHP: prüfen und HTML erzeugen. PDO: Datenbankzugriff. MySQL/MariaDB: Datensätze speichern. Die Person-Klasse bündelt hier nur die drei gelesenen Werte. [1–4]

Lesen ist nicht Schreiben

SELECT liest. `fetch(PDO::FETCH_ASSOC)` holt die nächste Zeile als Array; `false` beendet die Schleife. `new Person(...)` erzeugt das Objekt. INSERT schreibt einen neuen Datensatz. Ein Objekt allein ändert keine DB-Zeile. [2–4]

SQL und Werte getrennt halten

`prepare()` bekommt SQL mit Platzhaltern, `execute()` bekommt die Werte. Platzhalter nicht zusätzlich in Anführungszeichen setzen. Für die feste SELECT-Abfrage ohne Nutzereingaben verwenden wir `query()`. [3, 4]

Vor dem Speichern prüfen

`isset()` erkennt vorhandene, nicht-null Einträge, nicht deren Gültigkeit. `is_string()` fängt Arrays ab. `trim()` entfernt äußere Leerzeichen. Danach Pflichtfeld und Länge prüfen. `mb_strlen($text, "UTF-8")` zählt Zeichen; `mb_check_encoding()` prüft die Textkodierung. [9]

Erst bei der HTML-Ausgabe maskieren

Die DB speichert normale geprüfte Texte, keine vorab HTML-maskierten Namen. Der vorbereitete `html()`-Helper verwendet `htmlspecialchars()`. Auch Werte aus der DB werden bei der Anzeige maskiert. [6]

Rückleitung und Fehlermeldung

Vor `header()` keine Ausgabe; danach `exit`. HTTP 303 führt zu einer GET-Anfrage. Die Tabelle wird neu gelesen. Die bekannte Session überbrückt nur Meldungen und gültige Formwerte. F5 auf der GET-Seite erzeugt kein neues INSERT. [7]

Ein technischer Fehler ist kein leerer Datenbestand

`try/catch` fängt `PDOException` ab. Bei DB-Ausfall eine Meldung anzeigen, nicht „keine Personen“ behaupten. Zugangsdaten und rohe SQL-Fehler nicht im Browser ausgeben. [5]

Herkunft und Referenzen

BigPicture_PHP .pdf, S. 3–4: PDO, Validierung und HTML-Ausgabe. HTML-Kapitel 4: Tabellen und Formulare. Konkrete Personenverwaltung: neu erstellte didaktische Übertragung nach deinem Auftrag.

[1] PDO-DSN · [2] `PDOStatement::fetch` · [3] `PDO::prepare` · [4] `PDOStatement::execute` · [5] PDO-Fehlerbehandlung · [6] `htmlspecialchars` · [7] MDN: HTTP 303 · [8] MySQL: AUTO_INCREMENT · [9] `mb_strlen`. Vollständige Links stehen in QUELLEN.md und in der Webeinführung. Technischer Abgleich: 01.10.2026.